EX17 - Modulos e imports - Separar el proyecto en archivos

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex17_vertical_motion_modules.zip`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Separar la simulacion vertical en vertical_motion.py, data_processing.py y main.py para que cada archivo tenga una responsabilidad clara.

Variables / elementos del problema

Nombre	Significado
vertical_motion.py	fisica, validacion, simulacion y referencia
data_processing.py	postproceso de la trayectoria
main.py	configuracion y orquestacion

Ecuacion o logica en papel

```
main.py -> functions imported from vertical_motion.py and data_processing.py
```

Comprobacion manual

Los resultados fisicos no deben cambiar respecto a EX16. El cambio es de arquitectura: cada responsabilidad vive ahora en un modulo distinto.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- funciones
- diccionarios
- NumPy
- mascaras

Nuevo en este ejercicio:

- modulo = archivo .py
- from ... import ...
- dependencias por modulo
- def main()
- __name__
- main guard
- separacion de responsabilidades

3. Conceptos Python nuevos

1. Modulo

Sintaxis: `vertical_motion.py`

Un modulo es un archivo Python que puede definir funciones, clases o constantes reutilizables desde otros archivos.

2. Import interno

Sintaxis: `from vertical_motion import validate_inputs`

main.py importa una funcion concreta y luego puede llamarla directamente.

3. __name__

Sintaxis: `__name__`

Python asigna este nombre especial a cada modulo. Cuando un archivo se ejecuta directamente vale "__main__".

4. Main guard

Sintaxis: `if __name__ == "__main__": main()`

Ejecuta main() solo cuando este archivo se lanza directamente, no cuando se importa.

4. Codigo paso a paso

Paso 1. vertical_motion.py

```
def validate_inputs(config): ...
def simulate_vertical_motion(config): ...
def euler_step(...): ...
def analytical_solution(config): ...
```

Este archivo concentra el modelo y los calculos relacionados con el movimiento. Tambien importa sqrt y NumPy porque sus funciones los utilizan.

Paso 2. data_processing.py

```
def process_trajectory(trajectory_array, config):
    ...
    return processed_results, processed_data
```

Este modulo no controla el vuelo: recibe trajectory_array ya calculado y lo filtra, resume y amplia con energia.

Paso 3. Importar funciones en main.py

```
from vertical_motion import validate_inputs, simulate_vertical_motion, analytical_solution, calculate_errors
from data_processing import process_trajectory
```

main.py conecta los modulos sin copiar sus implementaciones.

Paso 4. Definir main()

```
def main():
    config = {
        "h0": 0.0,
        "v0": 50.0,
        "t0": 0.0,
        "g": 9.81,
        "dt": 0.1
    }
```

main() es el orquestador: prepara entradas y decide en que orden se llaman las funciones.

Paso 5. Flujo entre modulos

```
results, trajectory_array = simulate_vertical_motion(config)
processed_results, processed_data = process_trajectory(trajectory_array, config)
```

Los datos retornados por una funcion pasan a otra. La comunicacion se hace mediante parametros y retornos.

Paso 6. Proteger main

```
if __name__ == "__main__":
    main()
```

Al ejecutar main.py, la condicion es True. Si main.py se importara desde otro archivo, no arrancaria automaticamente el flujo.

5. Flujo de ejecucion

```
main.py
 -> validate_inputs (vertical_motion)
 -> simulate_vertical_motion
 -> analytical_solution / calculate_errors
 -> process_trajectory (data_processing)
 -> print results
```

6. Resultado / comprobacion esperada

Ejecutando main.py desde la carpeta del ejercicio se obtienen los mismos resultados de simulacion/postproceso que antes, ahora con responsabilidades separadas.

7. Errores tipicos al empezar

- Olvidar que cada modulo debe importar sus propias dependencias.
- Poner codigo de ejecucion top-level que se dispara al importar.
- Crear imports circulares.
- Confundir importar una funcion con ejecutarla.

8. Si solo recuerdas cinco cosas

- Un modulo es un archivo .py.
- import conecta archivos.
- main.py orquesta.
- Los datos viajan por parametros y retornos.
- El main guard evita ejecucion accidental al importar.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.